CREATE, INSERT, SELECT

```
CREATE DATABASE DBMS1_2025
DROP DATABASE DBMS1_2025
USE DBMS1_2025

--SYNTAX TO CREATE TABLE
-- CREATE TABLE TABLENAME
--(COL1     DATATYPE,
--COL2 DATATYPE);

 DROP TABLE STUDENT


CREATE TABLE STUDENT
(
      ROLLNO INT,
      NAME VARCHAR(20)
);


SELECT * FROM STUDENT
------------------HOW TO INSERT THE DATA?--------------------

--METHOD1: WITH COLUMN NAME

INSERT INTO STUDENT (ROLLNO,NAME) VALUES (1,'RAM');


INSERT INTO STUDENT(NAME,ROLLNO) VALUES ('SHIVAM',2);

INSERT INTO STUDENT(NAME,ROLLNO) VALUES (3,'XYZ');


INSERT INTO STUDENT (NAME) VALUES('MITALI');


SELECT * FROM STUDENT
--METHOD2: WITHOUT COLUMN NAME
--If you're inserting values for all columns in order, you can skip column names

INSERT INTO STUDENT VALUES (3,'KIRTI');


--ERROR
INSERT INTO STUDENT VALUES ('SHYAM',4)


--METHOD:3 INSERT MULTIPLE RECORD
--batch inserts
INSERT INTO STUDENT VALUES (5,'MAYA'),(6,'RESHMA'),(7,'MAYUR');
```

```sql
--INSERT WITH NULL VALUE

INSERT INTO STUDENT (NAME) VALUES('MITAL');


INSERT INTO STUDENT VALUES(8,NULL);

INSERT INTO STUDENT VALUES(NULL,NULL);

INSERT INTO STUDENT VALUES(9,'AMIT')

---------------HOW TO FETCH INSERTED DATA?-------------

--FATCH ALL THE INSERTED RECORDS

SELECT * FROM STUDENT


--SELECT SPECIFIC col

SELECT ROLLNO
FROM STUDENT

SELECT NAME
FROM STUDENT

SELECT ROLLNO,NAME
FROM STUDENT


-------------------SELECT WITH CONDITION-------------------------

SELECT ROLLNO, NAME
FROM STUDENT
WHERE ROLLNO>5

SELECT *
FROM STUDENT
WHERE NAME='RESHMA'

SELECT ROLLNO, NAME
FROM STUDENT
WHERE ROLLNO=5;

--------------------SELECT WITH NULL---------------------

--WRONG WAY
--IT WILL NOT RETURN ANY RECORD

SELECT * FROM
STUDENT
WHERE NAME = NULL

--NULL is not a value.
```

```
--it cannot be evaluated with the equality (=) operator.
-- it should be checked explicitly with the IS operator

SELECT * FROM
STUDENT
WHERE NAME IS NULL


----------------------SELECT TOP RECORDS-----------------------
SELECT *
FROM STUDENT

SELECT TOP 3 *
FROM STUDENT

SELECT TOP 10 NAME
FROM STUDENT

--PERCENT
SELECT TOP 10 PERCENT *
FROM STUDENT


--SQL Server rounds up, not down. So even though 11% of 10 rows is 1.1, it returns 2
rows.

SELECT TOP 11 PERCENT *
FROM STUDENT

---------------------------SELECT UNIQUE RECORD---------------------------
SELECT NAME
FROM STUDENT


SELECT DISTINCT NAME
FROM STUDENT

--BY DEFAULT DISTINCT DISPLAY THE RESULT IN ASCENDING ORDER

SELECT DISTINCT ROLLNO,NAME
FROM STUDENT


INSERT INTO STUDENT VALUES(7,'MAYUR')
INSERT INTO STUDENT VALUES(7,'KRISH')
SELECT *
FROM STUDENT

---------------OPERATORS------------------
--1. AND
--2. OR
--3. IN / NOT IN
--4. NOT
--5. BETWEEN
```

```
---AND---
SELECT *
FROM STUDENT
WHERE NAME='SHIVAM' AND ROLLNO=2

SELECT *
FROM STUDENT
WHERE ROLLNO>2 AND ROLLNO<6

---OR--
SELECT *
FROM STUDENT
WHERE  NAME='RAM' OR NAME='SHIVAM'


SELECT *
FROM STUDENT
WHERE ROLLNO=1 OR ROLLNO=2


SELECT *
FROM STUDENT
WHERE ROLLNO=1 OR ROLLNO=2 OR ROLLNO=3


-----IN/NOT IN------
SELECT *
FROM STUDENT
WHERE ROLLNO IN (1,2,3)

SELECT *
FROM STUDENT
WHERE ROLLNO NOT IN (1,2,3)

SELECT *
FROM STUDENT
WHERE NAME IN ('KIRTI','MAYA')


---NOT---
SELECT *
FROM STUDENT
WHERE NOT NAME='SHIVAM'


SELECT *
FROM STUDENT
WHERE  NAME <> 'SHIVAM'


SELECT *
FROM STUDENT
WHERE  NAME != 'SHIVAM'
```

```sql
---BETWEEN----
--BETWEEN IS INCLUSIVE
SELECT *
FROM STUDENT
WHERE ROLLNO BETWEEN 1 AND 5


SELECT *
FROM STUDENT
WHERE ROLLNO BETWEEN 5 AND 10;

-- WRITE QUERY TO DISPLAY STUDENT WHOSE NAME IS EITHER 'RAM' OR 'SHYAM' AND ROLL NO
BETWEEN 1 TO 10

SELECT *
FROM STUDENT
WHERE (NAME='RAM' OR NAME='SHYAM') AND (ROLLNO BETWEEN 1 AND 10)


---ODD/EVEN ROLL NO

SELECT *
FROM STUDENT
WHERE ROLLNO%2=0


SELECT *
FROM STUDENT
WHERE ROLLNO%2!=0

---RETRIVE WHOLE TABLE WITHOUT USING * --------
SELECT ROLLNO,NAME
FROM STUDENT
```

SELECTINTO, UPDATE, DELETE, ALTER, RENAME

```
USE DBMS1_2025

CREATE TABLE EMPLOYEE(
Name VARCHAR(50),
City VARCHAR(50),
Age INT);

INSERT INTO EMPLOYEE VALUES
('Jay Patel','Rajkot',30),
('Rahul Dave','Baroda',35),
('Jeet Patel','Surat',31),
('Vijay Raval','Rajkot',30);

SELECT *
FROM EMPLOYEE

--Create table EMPLOYEE_DETAIL from Employee with all the columns and data.

SELECT *
INTO EMPLOYEE_DETAIL
FROM EMPLOYEE

SELECT *
FROM EMPLOYEE_DETAIL

drop table EMPLOYEE_DETAIL

--Create table EMPLOYEE_DATA from Employee with first two columns with no data.

SELECT NAME,CITY
INTO EMPLOYEE_DATA
FROM EMPLOYEE
WHERE 1=0

SELECT *
FROM EMPLOYEE_DATA

---------------------WITH DIFFERENT cONDITION----------------------------------
SELECT * INTO EMPLOYEE_DATA1
FROM EMPLOYEE
WHERE 2+2=5

SELECT * FROM EMPLOYEE_DATA1
-----

SELECT * INTO EMPLOYEE_DATA2
FROM EMPLOYEE
WHERE 2+2=4

SELECT * FROM EMPLOYEE_DATA2
-------
SELECT * INTO Employee_info
```

```
FROM EMPLOYEE
WHERE 'A'='B'

SELECT * FROM Employee_info
--------
SELECT * INTO Employee_info1
FROM EMPLOYEE
WHERE 'ABC'='BCD'

SELECT *
FROM Employee_info1
----------
SELECT * INTO Employee_info2
FROM EMPLOYEE
WHERE 'A'='a'

SELECT *
FROM Employee_info2

SELECT * INTO Employee_info3
FROM EMPLOYEE
WHERE 'AB'='ab'
------------------------INSERT ----------------
--Insert the Data into Employee_info from Employee Whose CITY is Rajkot
SELECT * FROM Employee_info


INSERT INTO Employee_info
SELECT * FROM EMPLOYEE
WHERE CITY='RAJKOT';

select * from Employee_info

--Insert the Data into Employee_info from
--Employee Whose age is more than 32.
INSERT INTO Employee_info
SELECT * FROM EMPLOYEE
WHERE Age>32;


---CREATE EMP AND INSERT DATA WHOSE AGE >30 from Employee

SELECT * INTO EMP
FROM EMPLOYEE
WHERE AGE>30

SELECT * FROM EMP



--NOTE
--Indexes, constraints, and triggers defined in the source table
--are not transferred to the new table using SELECT into statement.
```

```
------------------------------UPDATE-----------------
SELECT * FROM EMPLOYEE


--Update age of employee named 'Jay Patel' to 32

UPDATE EMPLOYEE
SET AGE=32
WHERE NAME='Jay Patel'

--CHANGE CITY OF JAY PATEL TO MORBI


UPDATE EMPLOYEE
SET CITY='MORBI'
WHERE NAME='JAY PATEL'

SELECT * FROM EMPLOYEE

-- Update age to 33 for all employees from 'Rajkot'



UPDATE EMPLOYEE
SET AGE=33
WHERE CITY='RAJKOT'

--CHAGE NAME JEET PATEL TO JEET




UPDATE EMPLOYEE
SET NAME='JEET'
WHERE NAME='JEET PATEL'



-- Update name to jeet p patel and age to 34 of employee from city 'Surat'


UPDATE EMPLOYEE
SET Name = 'Jeet P. Patel', Age = 34
WHERE City = 'Surat';

--CHANGE NAME OF SURAT TO TEXTILE CITY

UPDATE EMPLOYEE
SET CITY='TEXTILE CITY'
WHERE CITY='SURAT'

SELECT * FROM EMPLOYEE
```

```sql
-- Increase age by 1 for employees in 'Rajkot' or 'Baroda'
UPDATE EMPLOYEE
SET Age=Age+1
WHERE City IN ('Rajkot', 'Baroda');

-- Set city to 'Ahmedabad'
--where age is 34 and city is 'TEXTILE CITY'

UPDATE EMPLOYEE
SET City = 'Ahmedabad'
WHERE Age = 34 AND City = 'TEXTILE CITY';

--SET AGE NULL
UPDATE EMPLOYEE
SET AGE=NULL
WHERE NAME='JAY PATEL'

--UPDATE SAME VALUE IN ALL RECORD
UPDATE EMPLOYEE
SET AGE=25

SELECT *
FROM EMPLOYEE

------------------------DELETE------------------------
CREATE TABLE EMPLOYEE(
Name VARCHAR(50),
City VARCHAR(50),
Age INT);

DELETE from
EMPLOYEE


INSERT INTO EMPLOYEE VALUES
('Jay Patel','Rajkot',30),
('Rahul Dave','Baroda',35),
('Jeet Patel','Surat',31),
('Soham Raval','Valsad',30),
('Meet Shah','Baroda',35),
('Milan Patel','Vapi',31),
('Kiran Manek','Ahmedabad',30),
('Mitesh Patel','Kalol',NULL);

--DELETE EMPLOYEE FROM RAJKOT


DELETE FROM EMPLOYEE
WHERE CITY='RAJKOT'

select * from EMPLOYEE
--REMOVE THE RECORD WHOSE AGE IS 30

DELETE FROM EMPLOYEE
```

```
WHERE AGE=30

SELECT * FROM EMPLOYEE

--REMOVE RECORD WHOSE CITY IS BARODA AND AGE IS 35
DELETE FROM EMPLOYEE
WHERE CITY='BARODA' AND AGE=35

-- DELETE WITH IN AND OR OPERATOR
DELETE FROM EMPLOYEE
WHERE CITY IN ('VAPI','AHMEDABAD')


DELETE FROM EMPLOYEE
WHERE CITY='VAPI'OR CITY='AHMEDABAD'

--DELETE where Age is NULL
DELETE FROM EMPLOYEE
WHERE AGE IS NULL

--DELETE ALL RECORDS
DELETE FROM
EMPLOYEE

select * from EMPLOYEE

-----------ALTER-------------------
--ALTER  is used to add, modify, or drop columns in a table

SELECT * FROM EMPLOYEE

DROP TABLE EMPLOYEE

CREATE TABLE EMPLOYEE(
Name VARCHAR(50),
City VARCHAR(50),
Age INT);

INSERT INTO EMPLOYEE VALUES
('Jay Patel','Rajkot',30),
('Rahul Dave','Baroda',35),
('Jeet Patel','Surat',31),
('Soham Raval','Valsad',30),
('Meet Shah','Baroda',35),
('Milan Patel','Vapi',31),
('Kiran Manek','Ahmedabad',30),
('Mitesh Patel','Kalol',NULL);

--ADD COLUMNS

---ADD COLUMN SALARY

ALTER TABLE EMPLOYEE
ADD SALARY DECIMAL(8,2);
```

```
SELECT * FROM EMPLOYEE

--ADD COLUMNS DEPARTMENT AND DESIGNATION
select *
from EMPLOYEE

ALTER TABLE EMPLOYEE
ADD DEPARTMENT VARCHAR(20),DESIGNATION VARCHAR(20)

--ADD COLUMN MOBILE AND ADDRESS
ALTER TABLE EMPLOYEE
ADD MOBILE VARCHAR(20), ADDRESS VARCHAR(50)

SELECT * FROM EMPLOYEE

-----------
update employee
set salary=123.45
where age>=30

---- DROP COLUMN

ALTER TABLE EMPLOYEE
DROP COLUMN MOBILE

SELECT * FROM EMPLOYEE

--DROP MULTIPLE COULMNS
ALTER TABLE EMPLOYEE
DROP COLUMN DESIGNATION,ADDRESS


select *
from employee


---------MODIFY COLUMN
ALTER TABLE EMPLOYEE
ALTER COLUMN CITY CHAR(30)


ALTER TABLE EMPLOYEE
ALTER COLUMN NAME CHAR(20)

select *
from EMPLOYEE

ALTER TABLE EMPLOYEE
ALTER COLUMN AGE DECIMAL(5,2)


SELECT * FROM EMPLOYEE
```

```sql
--LEAD TO DATA LOSS
ALTER TABLE EMPLOYEE
ALTER COLUMN SALARY INT


--ERROR
ALTER TABLE EMPLOYEE
ALTER COLUMN CITY INT



--CHANGE AGE TO VARCHAR (Will work, but Wrong Practice)

ALTER TABLE EMPLOYEE
ALTER COLUMN AGE VARCHAR(10)

SELECT * FROM EMPLOYEE

drop table EMPLOYEE

---IMPLICIT TYPE CASTING FROM VARCHAR DATA TYPE TO INT DONE BY SQL SERVER
UPDATE EMPLOYEE
SET AGE=AGE+5
WHERE NAME='jeet patel'

INSERT INTO EMPLOYEE VALUES ('SOHAM','RAJKOT','A')

SELECT * FROM
EMPLOYEE

UPDATE EMPLOYEE
SET AGE=AGE+5
WHERE NAME='SOHAM'


--ADD MOBILE COLUMN
ALTER TABLE EMPLOYEE
ADD MOBILE decimal(8,2)

SELECT * FROM EMPLOYEE

SP_HELP EMPLOYEE

-----------------RENAME------------
--sp_rename system stored procedure lets you rename
--various database objects—including tables, columns

SELECT *
FROM EMPLOYEE



--RENAME COLUMN MOBILE TO CONTACTNO
```

```
SP_RENAME 'EMPLOYEE.MOBILE','CONTACTNO'

--WHY WARNING??????????????


--When you rename a table, column, or any object,
--SQL Server doesn't update references inside
--stored procedures, views, scripts, functions, triggers, or other modules
--That message is Microsoft's way of reminding you to check and fix dependent code
manually.

--RENAME TABLE NAME
SP_RENAME 'EMPLOYEE','EMPLOYEE_MASTER'


--CHANGE THE NAME EMPLOYEE_MASTER TO EMP_INFO
SP_RENAME 'EMPLOYEE_MASTER','EMP_INFO'

SELECT * FROM EMP_INFO

-------TRUNCATE-----------
--TRUNCATE deletes all rows in one go, not one by one row



TRUNCATE TABLE EMP_INFO

select * from emp_info


--DROP

DROP TABLE EMP_INFO

DROP DATABASE DEMO


DROP TABLE EMPLOYEE
------------------------------
CREATE TABLE EMPLOYEE(
Name VARCHAR(50),
City VARCHAR(50),
Age INT);

INSERT INTO EMPLOYEE VALUES
('Jay Patel','Rajkot',30),
('Rahul Dave','Baroda',35),
('Jeet Patel','Surat',31),
('Soham Raval','Valsad',30),
('Meet Shah','Baroda',35),
('Milan Patel','Vapi',31),
('Kiran Manek','Ahmedabad',30),
('Mitesh Patel','Kalol',NULL);
```

```sql
--DIFFERANCE---


select * from employee

--DELETE WITH CONDITION
DELETE FROM
EMPLOYEE
WHERE CITY='RAJKOT'

SELECT * FROM EMPLOYEE

---DELETE ALL RECORDS
delete from EMPLOYEE



---DELETE ALL RECORDS USING TRUNCATE
TRUNCATE TABLE
EMPLOYEE

SELECT *
FROM EMPLOYEE
--STRUCTURE OF TABLE IS STILL EXIST


--DROP
DROP TABLE EMPLOYEE
```

LIKE OPERATOR

```sql
USE DBMS1_2025

CREATE TABLE STUDENTS(
STUID INT,
FIRSTNAME VARCHAR(25),
LASTNAME VARCHAR(25),
WEBSITE VARCHAR(50),
CITY VARCHAR(25)
);

INSERT INTO STUDENTS VALUES(1011,'KEYUR','PATEL','ECHONTHENET.COM','RAJKOT');
INSERT INTO STUDENTS VALUES(1022,'HARDIK','SHAH','DIGMINECRAFT.COM','AHMEDABAD');
INSERT INTO STUDENTS VALUES(1033,'KAJAL','TRIVEDI','BIGACTIVITIES.COM','BARODA');
INSERT INTO STUDENTS VALUES(1044,'BHOOMI','GAJERA','CHECKYOURMATH.COM','AHMEDABAD');
INSERT INTO STUDENTS VALUES(1055,'HARMIT','MITEL',NULL,'RAJKOT');
INSERT INTO STUDENTS VALUES(1066,'ASHOK','JANI',NULL,'BARODA');

--------------- STARTS WITH-------------------
select *
from STUDENTS


--NAME STARTS WITH 'H'

SELECT *
FROM STUDENTS
WHERE FIRSTNAME LIKE 'H%'



--NAME STARTS WITH 'A'

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE 'A%'


--NAME STARTS WITH 'T'
SELECT *
FROM STUDENTS
WHERE LASTNAME LIKE 't%'



--ENDS WITH

--NAME ENDS WITH 'K'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '%K'
```

```
--LAST NAME ENDS WITH 'L'


SELECT LASTNAME
FROM STUDENTS
WHERE LASTNAME LIKE '%L'



--CITY ENDS WITH 'bad'


SELECT DISTINCT CITY
FROM STUDENTS
WHERE CITY LIKE '%BAD'

--CONTAINS

--CONTAINS HA

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '%HA%'



--FIRST NAME CONTAINS R


SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '%R%'


--WEBSITE CONTAINS .COM

SELECT WEBSITE
FROM STUDENTS
WHERE WEBSITE LIKE '%.COM%'


--------------POSITIONS AND LENGTH--------------------

--FIRST NAME OF LENGTH 5

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_____'


--START WITH 'K' AND HAVING 5 LETTER

SELECT FIRSTNAME
```

```sql
FROM STUDENTS
WHERE FIRSTNAME LIKE 'K____'

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_____' and FIRSTNAME LIKE 'K%'

---SECOND CHAR IS 'A'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_A%'


--SECOND CHAR 'A' AND LAST CHAR 'L'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_A%L'

--STARTS WITH 'H' AND ENDS WITH 'K'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE 'H%K'

--LENGTH 6 AND LAST CHAR 'I'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_____I'



--NAME STARTS WITH EITHER  'A','B', OR 'C'
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '[A,B,C]%'

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE 'A%'
OR FIRSTNAME LIKE 'B%'
OR FIRSTNAME LIKE 'C%'

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '[A-C]%'

--NAME STARTS WITH A TO H

SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '[A-H]%'
```

```sql
--CONTAINS VOWEL IN THEIR NAME
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '%[A,E,I,O,U]%'



--SECOND CHAR VOWEL
SELECT FIRSTNAME
FROM STUDENTS
WHERE FIRSTNAME LIKE '_[A,E,I,O,U]%'

--STUDENT ID ENDS WITH NUMBER
SELECT STUID
FROM STUDENTS
WHERE STUID LIKE '%[0-9]'

SELECT * FROM STUDENTS

--WEBSITE CONTAIN ANY NON DIGIT
SELECT WEBSITE
FROM STUDENTS
WHERE WEBSITE LIKE '%[^0-9]%'

---not like
-- name not starts with K
SELECT FIRSTNAME
FROM STUDENTS
WHERE
FIRSTNAME NOT LIKE 'K%'

---not like
--^ SHOULD BE IN BRACKET
SELECT FIRSTNAME
FROM STUDENTS
WHERE
FIRSTNAME  LIKE '[^K]%'

--WRONG WAY
SELECT FIRSTNAME
FROM STUDENTS
WHERE
FIRSTNAME  LIKE '^[K]%'
```

AGGREGRATE FUNCTION

```sql
USE DBMS1_2025

CREATE TABLE PRODUCT
(ProductID  INT,
ProductName VARCHAR(20),
PRICE DECIMAL(20,6));


INSERT INTO PRODUCT VALUES (1,'PHONE',37000),
(2,'TV',25000),
(3,'LAPTOP',70000),
(4,'TABLET',45000),
(5,'PENDRIVE',400),
(6,'DESKTOP',50000),
(7,'DESKTOP',50000),
(NULL,'HARDDISK',6000)
INSERT INTO PRODUCT VALUES (1,'smartPHONE',57000)

SELECT * FROM PRODUCT


--FIND MAXIMUM PRICE PRODUCTS

SELECT MAX(PRICE)
FROM PRODUCT

--GIVE ALIAS TO RESULTANT COLUMN
--alias is a temporary name given to a column
--within the scope of a single query.

--4 DIFFERENT WAYS TO GIVE ALIAS

--display maximum price
--1
SELECT MAX(PRICE) AS MAX_PRICE
FROM PRODUCT

--2
SELECT MAX(PRICE) AS 'MAX_PRICE'
FROM PRODUCT

--3
SELECT MAX(PRICE) AS [MAX_PRICE]
FROM PRODUCT

--4
SELECT MAX(PRICE) MAX_PRICE
FROM PRODUCT

--display minimum price
SELECT MIN(PRICE) AS MINimum
```

```sql
FROM PRODUCT



--display average price
SELECT AVG(PRICE) AS AVERAGE_PRICE
FROM PRODUCT



--display total of all product's price
SELECT SUM(PRICE) AS TOTAL_PRICE
FROM PRODUCT


SELECT * FROM PRODUCT

--COUNT TOTAL PRODUCTS

--* INCLUDES NULL
--COUNTS NUMBER OF ROWS


SELECT COUNT(*) AS TOTAL_PRODUCT
FROM PRODUCT


--EXCLUDE NULL
--COUNT(column_name):counts only rows where that specific column is non-NULL

SELECT COUNT(ProductID) AS TOTAL_PRODUCT
FROM PRODUCT

--WITH WHERE CONDITION
--count the product whose price is more then 5000


SELECT COUNT(ProductID) AS COUNT
FROM Product
WHERE Price > 5000;


SELECT COUNT(ProductNAME) AS ProductCOUNT
FROM Product
WHERE Price > 5000;

--find the TOTAL value of the products
--whose price is less the 4000
SELECT SUM(PRICE)
FROM PRODUCT
WHERE PRICE<4000
```

```
--

------MULTIPLE FUNCTIONS---------
SELECT
      MAX(PRICE) AS MAX_PRICE,
      MIN(PRICE) AS MIN_PRICE,
      AVG(PRICE) AS AVG_PRICE
FROM Product


--Return all products with a higher price than the average price:


select avg(price) from product
--ANS:37822.222222

select * from product
where price>
(select avg(price) from product)

--select * from product where price>(37822.222222)


select * from product

---distinct--


SELECT COUNT(ProductName)AS COUNT FROM PRODUCT

SELECT COUNT(DISTINCT ProductName) AS UNIQUE_COUNT FROM PRODUCT
```

GROUP BY, HAVING

```sql
USE DBMS1_2025

CREATE TABLE STUDENT_INFO
(Rno INT,
Name VARCHAR(10),
Branch VARCHAR(10),
Semester INT,
CPI DECIMAL(4,2),
CITY VARCHAR(20)
)

INSERT INTO STUDENT_INFO
VALUES
(101,'eRamesh','CE',3,9,'RAJKOT')
(101,'Ramesh','CE',3,9,'RAJKOT'),
(102,'Mahesh','EC',3,8,'BARODA'),
(103,'Suresh','ME',4,7,'AHMEDABAD'),
(104,'Amit','EE',4,8,'AHMEDABAD'),
(105,'Anita','CE',4,8,'RAJKOT'),
(106,'Reeta','ME',3,7,'BARODA'),
(107,'Rohit','EE',4,9,'AHMEDABAD'),
(108,'Chetan','CE',3,8,'RAJKOT'),
(109,'Rakesh','CE',4,9,'RAJKOT')

SELECT * FROM STUDENT_INFO



--MAX CPI
SELECT MAX(CPI) FROM STUDENT_INFO

--1) BRANCH WISE MAX CPI

SELECT MAX(CPI) AS MAX_CPI
FROM STUDENT_INFO
GROUP BY BRANCH

--WITH COLUMN SELECTION
SELECT BRANCH,MAX(CPI) AS MAX_CPI
FROM STUDENT_INFO
GROUP BY BRANCH


--ERROR
--In SQL Server, you cannot select a column that’s not in the
--GROUP BY clause unless It’s wrapped in an aggregate function


---CAN'T SELECT CITY COLUMN WHEN GROUPING IS ON THE BRANCH
SELECT CITY,MAX(CPI) AS MAX_CPI
FROM STUDENT_INFO
GROUP BY BRANCH
```

```sql
--2) BRANCH WISE HIGHEST,LOWEST AND AVERAGE CPI

SELECT
      BRANCH,
      MAX(CPI) AS MAX_CPI,
      MIN(CPI) AS MIN_CPI,
      AVG(CPI) AS AVG_CPI
FROM STUDENT_INFO
GROUP BY BRANCH


--3) COUNT BRANCH WISE STUDENTS

SELECT
      BRANCH,
      COUNT(NAME) as Stu_Count
FROM STUDENT_INFO
GROUP BY BRANCH

--4) COUNT CITY WISE STUDENTS
SELECT
      CITY,
      COUNT(NAME) as Stu_Count
FROM STUDENT_INFO
GROUP BY CITY

---5) SEM WISE AVG CPI
SELECT
      Semester,
      AVG(CPI) AS AVERAGE_CPI
FROM STUDENT_INFO
GROUP BY Semester

--------------------------------CONDITION---------------------------------
---DATA FILTERATION BEFORE GROUPING
--6) Branch WISE Count of Students   (Only Semester 4)

SELECT BRANCH,COUNT(NAME) as Count_Branchwise
FROM STUDENT_INFO
WHERE SEMESTER=4
GROUP BY Branch

--7) Count of Students per Semester (Only for CE Branch)


SELECT Semester, COUNT(*) AS Student_Count
FROM STUDENT_INFO
WHERE Branch = 'CE'
GROUP BY Semester;

--8) BRANCH WISE MIN CPI , ONLY FOR SEM 3
```

```
SELECT BRANCH,MIN(CPI)
FROM STUDENT_INFO
WHERE Semester=3
GROUP BY BRANCH

---------------DATA FILTERATION AFTER GROUPING-----
--to filter groups based on specified conditions
--HAVING is used


--9)count sem wise students and display only sem
--where sem wise count is more then 4


--ERROR--CAN'T USE AGGREGRATE FUNCTIONS DIRECTLY WITH WHERE

--Use HAVING only when:
--You need to filter based on aggregate values (e.g., SUM(), MIN(), COUNT()) after
grouping.

--error
SELECT
      COUNT(NAME) AS STUDENT_COUNT,
      Semester
FROM STUDENT_INFO
WHERE COUNT(NAME)>4
GROUP BY Semester

-----CORRECT
SELECT
      COUNT(NAME) AS STUDENT_COUNT,
      Semester
FROM STUDENT_INFO
GROUP BY Semester
HAVING COUNT(NAME)>4


--10)
--Find out All the Branches with avg CPI,
--whose avg CPI is more than 8.

--ERROR--CAN'T USE AGGREGRATE WITH WHERE
SELECT BRANCH,AVG(CPI) AS [MAX_CPI]
FROM STUDENT_INFO
WHERE AVG(CPI)>8
GROUP BY BRANCH

-----CORRECT
SELECT BRANCH,AVG(CPI) AS [MAX_CPI]
FROM STUDENT_INFO
GROUP BY BRANCH
HAVING AVG(CPI)>8

-----------DATA FILTERATION BEFORE AND AFTER GROUPING------
```

```
--11)semster wise total student for 'CE' and 'EC' branch
--display only those where total students per branch is more then 1

SELECT SEMESTER, COUNT(RNO) AS TOTAL_STUDENT
FROM STUDENT_INFO
WHERE BRANCH IN ('CE','EC')
GROUP BY SEMESTER
HAVING COUNT(RNO)>1


---------GROUPING ON MULTIPLE COLUMN---------
--12)
--BRANCH WISE, SEM WISE MIN CPI
SELECT SEMESTER,BRANCH, MIN(CPI)AS MIN_CPI
FROM STUDENT_INFO
GROUP BY SEMESTER,BRANCH


SELECT BRANCH,SEMESTER, MIN(CPI)AS MIN_CPI
FROM STUDENT_INFO
GROUP BY BRANCH,SEMESTER


---ORDER BY
-----------------------------------------------------------------
-- SELECT → WHERE → GROUP BY → HAVING → ORDER BY
-----------------------------------------------------------------

--NO SORTING
SELECT CPI from STUDENT_INFO

--SORTING ON CPI COLUMN
SELECT CPI
FROM STUDENT_INFO
ORDER BY CPI

SELECT CPI
FROM STUDENT_INFO
ORDER BY CPI ASC

--DESCENDING
SELECT CPI
FROM STUDENT_INFO
ORDER BY CPI DESC


--13)SELECT BRANCH WISE MAXIMUM CPI IN ASCENDING ORDER

SELECT BRANCH,MAX(CPI) as Branchwise_MaxCPI
FROM STUDENT_INFO
GROUP BY BRANCH
ORDER BY MAX(CPI)
```

```sql
--14)SELECT BRANCH WISE MAXIMUM CPI IN DESCDENDING ORDER
SELECT BRANCH,MAX(CPI) BRANCH_MAX_CPI
FROM STUDENT_INFO
GROUP BY BRANCH
ORDER BY MAX(CPI) DESC



--15)DISPLAY SEM WISE AVG SPI IN DESCENDING ORDER
SELECT SEMESTER,(AVG(CPI)) SEMwise_AVG_CPI
FROM STUDENT_INFO
GROUP BY SEMESTER
ORDER BY AVG(CPI) DESC

--16)
--Find out semester wise total students &
--arrange them in order with their count.
Select Semester, Count(Rno) AS [Total]
From STUDENT_INFO
Group By Semester
Order By Count(Rno)

--17)
--find Branch wise total students in descending order
SELECT Branch,COUNT(RNO) AS TOTAL
FROM STUDENT_INFO
GROUP BY BRANCH
ORDER BY  COUNT(RNO) DESC


--18)
--Find out Branch wise & Semester wise
--CE branch's students whose
--AVERAGE CPI is greater than 7.
--Do arrange the result in descending order to semester.

---POSIBLE SOLUTIONS
--1)
Select Branch, Semester, AVG(CPI) AS [AVG] From Student_INFO
Where Branch='CE'
Group By Branch, Semester
Having AVG(CPI) > 7
Order By Semester Desc

--2)
---EFFICIENT WAY
Select  Semester, AVG(CPI) AS [AVG] From Student_INFO
Where Branch='CE'
Group By  Semester
Having AVG(CPI) > 7
Order By Semester Desc

--3)
Select Branch, Semester, AVG(CPI) AS [AVG]
```

```
From Student_info
Group By Branch, Semester
Having AVG(CPI) > 7 and Branch='CE'
Order By Semester Desc



--19) finds the branch with the highest average CPI
SELECT  TOP 1 branch,avg(cpi) AS avg_cpi
FROM Student_info
GROUP BY branch
ORDER BY avg(cpi) DESC;



-----REVISION-----
--1) QUERY WITH NO FILTERING
SELECT BRANCH,AVG(CPI) AS avg_CPI
FROM STUDENT_INFO
GROUP BY BRANCH


--2) FILTERING WITH WHERE BEFORE GROUPING
SELECT BRANCH,avg(CPI) AS avg_CPI
FROM STUDENT_INFO
WHERE BRANCH IN ('CE','EC')
GROUP BY BRANCH

--3) FILTERING WITH HAVING AFTER GROUPING
SELECT BRANCH,avg(CPI) AS avg_CPI
FROM STUDENT_INFO
GROUP BY BRANCH
having avg(cpi)>7

--4) FILTERING WITH WHERE BEFORE GROUPING AND
--FILTERING WITH HAVING AFTER GROUPING
SELECT BRANCH,avg(CPI) AS avg_CPI
FROM STUDENT_INFO
WHERE BRANCH IN ('CE','EC')
GROUP BY BRANCH
having avg(cpi)>7
```

BUILT-IN FUNCTIONS

```
USE DBMS1_2025

------------1) MATHS FUNCTIONS--------------
--1) ABS()
--RETURNS ABSOLUTE VALUE
SELECT ABS(20), ABS(-50)



--2)CEILING()
--Returns the smallest integer value that is >= a number
SELECT CEILING(25.7),CEILING(25.3),CEILING(-13.5)



--3) FLOOR()
--Returns the largest integer value that is <= to a number
SELECT  FLOOR(25.75),FLOOR(-13.5)


--4) PI()
--Returns the value of PI
SELECT PI() as PI_Value

--5) POWER()
--Returns the value of a number raised to the power of another number
SELECT POWER(4, 2)


--6) ROUND()
--Rounds a number to a specified number of decimal places
SELECT ROUND(235.415, 0),ROUND(235.415, 1),ROUND(235.465, 2)

--7) SIGN()
--Returns the sign of a number
--(If number > 0, it returns 1,
--If number = 0, it returns 0,
--If number < 0, it returns -1)
SELECT SIGN(-12), SIGN(12), SIGN(0)


--8) SQRT()
--Returns the square root of a number
SELECT SQRT(64)

--9) SQUARE()
--Returns the square of a number
SELECT SQUARE(8)

------------2)STRING FUNCTIONS----------
--1) ASCII()
--The ASCII() returns the ASCII code value of the leftmost character of the character
expression.
```

```
SELECT ASCII('A'), ASCII('a'),ASCII('AB')


--2) CONCAT()
--CONCAT() function is used To join two or more strings into one
SELECT CONCAT('Darshan','University')


--3) CONCAT_WS()
--concatanation with seperator
SELECT CONCAT_WS(',','Darshan','University')

SELECT CONCAT_WS('_','Darshan','University')

SELECT CONCAT_WS('#123','Darshan',',','University')

SELECT CONCAT_WS('#123','Darshan','University','abc')

--4) CHARINDEX()
--return the index of a specific string expression within a given string.
SELECT CHARINDEX('t', 'Customer')

SELECT CHARINDEX('World','Hello World')

SELECT CHARINDEX('a','Hello World')


--5) LEFT(), RIGHT()
--used to extract a specific number of characters from the left-side or right-side of
a string.
SELECT LEFT('Darshan University',5)
SELECT RIGHT('Darshan University',5)



--6) LOWER(), UPPER()
SELECT LOWER('Darshan')
SELECT UPPER('Darshan')

--7) LTRIM(), RTRIM()
--used to remove additional spaces from the left side or right side of an input
string.
SELECT RTRIM('Darshan  ') + LTRIM('    University')
SELECT ('Darshan  ') + ('    University')

select TRIM('    DARSHAN     UNIVERSITY      ')

SELECT TRIM('r' FROM 'rrrrSQL Serverrrrr')

--8)STRING_SPLIT()
--It is a table-valued function that splits a string into a
--table that consists of rows of substrings based on a specified separator.
SELECT value FROM STRING_SPLIT('red,green,,blue', ',')
```

```
--9) REPLACE()
--To replace all occurrences of a substring within a string with a new substring
SELECT REPLACE('Darshan Institute','Institute','University')


--10)REPLICATE()
--It repeats a string a specified number of times.
SELECT REPLICATE('Darshan',3)

--11) REVERSE()
--It accepts a string argument and returns the reverse order of that string.
SELECT REVERSE('Darshan')

--12) SPACE()
--It returns a string of repeated spaces.
SELECT  SPACE(5)+ 'Darshan'


--13) SUBSTRING()
--It extracts a substring with a specified length
--starting from a location in an input string.
--Syntax: SUBSTRING(input_string, Start, Length);
SELECT SUBSTRING('SQL Server SUBSTRING', 5, 6)


--14)LEN()
--the number of characters in a string without including trailing spaces.
Select LEN('Darshan University')
Select LEN('Darshan University     ')
Select LEN('    Darshan University')

------------3) OTHER FUNCTIONS--------------------
--1)CAST()
--It converts a value (of any type) into a specified datatype.
SELECT CAST(25.65 AS varchar)

SELECT CAST(25.65 AS int)
--error
select cast('abc' as int)



--2) CONVERT()
SELECT CONVERT(varchar,25.65)

SELECT CONVERT(int,25.65)
--error
SELECT CONVERT(int,'abc')



-----------4) DATE TIME FUNCTIONS-------------
```

```sql
--1 GETDATE()
SELECT GETDATE() AS CURRENTDATE

--2) DAY()
--DAY accepts a date, datetime, or valid date string and
--returns the Day part as an integer value

SELECT DAY(GETDATE()) AS [Day]


--ISO Standard (safe and recommended)
SELECT DAY('2025-01-05') AS [Day]

SELECT DAY('2025-07-2 15:46:19.27')  AS [Day]

--YYYYMMDD (no dashes)
SELECT DAY('20250702')

--Named Month (if language setting supports it)
SELECT DAY('July 2, 2025')
SELECT DAY('1-JAN-2025')

--US Format (language-dependent)

SELECT DAY('07/02/2025') --mm/dd/yyyy



--3 MONTH()
SELECT MONTH(GETDATE()) AS MONTH

SELECT MONTH('20250105') AS [MONTH]

SELECT MONTH('2025-07-2 15:46:19.277')  AS [MONTH]


--4 YEAR()

SELECT YEAR(GETDATE()) AS [YEAR]

SELECT YEAR('20250105') AS [YEAR]

SELECT YEAR('2025-07-2 15:46:19.277')  AS [YEAR]

--5) DATEPART()
--It returns an integer corresponding to the
--datepart specified in DATEPART function.
SELECT DATEPART(YEAR, GETDATE())   AS 'Year';
SELECT DATEPART(MONTH, GETDATE())  AS 'Month';
SELECT DATEPART(DAY, GETDATE())    AS 'Day';
SELECT DATEPART(WEEK, GETDATE())   AS 'Week';
SELECT DATEPART(HOUR, GETDATE())   AS 'Hour';
SELECT DATEPART(MINUTE, GETDATE()) AS 'Minute';
SELECT DATEPART(SECOND, GETDATE()) AS 'Second';
```

```sql
--6) DATENAME()
--It returns a string corresponding to the datepart specified for the given date
SELECT DATENAME(MONTH, GETDATE())  AS 'Month';


--7)EOMONTH()
--The date function EOMONTH accepts a
--date, datetime, or valid date string and returns
--the end of month date

SELECT EOMONTH(GETDATE())


SELECT EOMONTH('2025-08-01')


SELECT
EOMONTH(GETDATE()) as 'End Of Current Month',
EOMONTH(GETDATE(),-1) as 'End Of Previous Month',
EOMONTH(GETDATE(),3) as 'End Of Month after 3 month';

--8) DATEADD()
--It returns datepart with added interval as a datetime.


--day
SELECT DATEADD(d, 1, '2025-07-4 15:15:20') AS ADDEDDATE


SELECT DATEADD(d, 1, getdate()) AS ADDEDDATE
SELECT DATEADD(dd, 1, '2025-07-4 15:15:20') AS ADDEDDATE
SELECT DATEADD(day, 1, '2025-07-4 15:15:20') AS ADDEDDATE

--month

SELECT DATEADD(m, 1, '2025-07-4 15:15:20') AS ADDEDDATE
SELECT DATEADD(mm, 1, '2025-07-4 15:15:20') AS ADDEDDATE
SELECT DATEADD(month, 1, '2025-07-4 15:15:20') AS ADDEDDATE

--year
SELECT DATEADD(yy, 1, '2025-07-4 15:15:20') AS ADDEDDATE
SELECT DATEADD(yyyy, 1, '2025-07-4 15:15:20') AS ADDEDDATE
SELECT DATEADD(year, 1, '2025-07-4 15:15:20') AS ADDEDDATE


--9) DATEDIFF() :
--difference between two dates
SELECT DATEDIFF(MINUTE, '2025-07-13', '2025-07-14')

SELECT DATEDIFF(HOUR, '2025-07-13', '2025-07-14')

SELECT DATEDIFF(DAY, '2025-07-01', '2025-07-14')
```

```sql
SELECT DATEDIFF(MONTH, '2025-07-01', '2025-08-14')

SELECT DATEDIFF(YEAR, '2024-07-01', '2025-08-14')

--10)ISDATE() :
--To check a string to see if it is a valid Date or Datetime field.
--ISDATE return 1 if true or 0 if false.


SELECT ISDATE('2025-01-01') as 'Valid';
SELECT ISDATE('2025-13-01') as 'Not Valid';

---NESING OF FUNCTIONS--
SELECT UPPER(TRIM('  hello world  '));


SELECT SUBSTRING(UPPER('hello world'), 1, 5);



---------INFORMATION-----------
--Format          Examples
--yyyy-MM-dd      2025-07-02  ISO STANDARD, unambiguous
--MM/dd/yyyy      07/02/2025  US style (July 2, 2025)
--dd/MM/yyyy      02/07/2025  UK/India style (2 July 2025)

SELECT
  FORMAT(GETDATE(), 'yyyy-MM-dd') AS IsoDate,
  FORMAT(GETDATE(), 'MM/dd/yyyy') AS UsDate,
  FORMAT(GETDATE(), 'dd/MM/yyyy') AS UkDate;

--'MM' – month
--'mm' – minutes
```

CONSTRAINTS

```
USE DBMS1_2025

-----1) NOT NULL CONSTRAINT------------
CREATE TABLE PERSON(
ID INT NOT NULL,
FIRSTNAME VARCHAR(255) NOT NULL,
AGE INT );



INSERT INTO PERSON VALUES(1,'GEETA',NULL);

SELECT * FROM PERSON
--ERROR
INSERT INTO PERSON VALUES(NULL,'MEETA',30);

INSERT INTO PERSON VALUES(2,NULL,30);

--Create a NOT NULL constraint on the "Age" column
--when the "Persons" table is already created.

ALTER TABLE PERSON
ALTER COLUMN AGE INT NOT NULL;

--WHY ERROR?
--BECAUSE EXISTING TABLE HAVE A RECORD WITH NULL AGE

DELETE FROM
PERSON
WHERE AGE IS NULL

--NOW,  NOT NULL CONSTRAINT ON AGE COLUMN
ALTER TABLE PERSON
ALTER COLUMN AGE INT NOT NULL;

--ERROR
INSERT INTO PERSON VALUES(1,'SHYAM',NULL)

---VALID
INSERT INTO PERSON VALUES(3,'MEERA',25)


----------------2) CHECK CONSTRAINT--------------
 CREATE TABLE PERSONS (
    ID INT ,
    LASTNAME VARCHAR(255),
    FIRSTNAME VARCHAR(255),
    AGE INT CHECK (AGE >= 18)
);


--ERROR
```

```sql
INSERT INTO PERSONS VALUES (1,'PATEL','MEET',12)

--VALID
INSERT INTO PERSONS VALUES (1,'PATEL','JEET',25)


DROP TABLE PERSONS

 CREATE TABLE PERSONS (
    ID INT ,
    LASTNAME VARCHAR(255),
    FIRSTNAME VARCHAR(255),
    AGE INT
);

--Create a CHECK constraint on the "Age" column
--when the "Persons" table is already created.

ALTER TABLE PERSONS
ADD CHECK (AGE>=18);

--ERROR
INSERT INTO PERSONS VALUES (1,'SHAH','NISHIT',12);


----------3) DEFAULT CONSTRAINT--------------
CREATE TABLE PERSON_INFO (
    ID INT ,
    LASTNAME VARCHAR(255) ,
    FIRSTNAME VARCHAR(255),
    AGE INT DEFAULT 18
);

INSERT INTO PERSON_INFO (ID,LastName,FirstName)
VALUES (1,'PATEL','SEEMA')

SELECT * FROM PERSON_INFO

--Create a DEFAULT constraint on the "Age" column
--when the "PERSON_INFO" table is already created.
DROP TABLE PERSON_INFO

CREATE TABLE PERSON_INFO (
    ID INT ,
    LASTNAME VARCHAR(255) ,
    FIRSTNAME VARCHAR(255),
    AGE INT
);

ALTER TABLE PERSON_INFO
ADD CONSTRAINT df_Age
DEFAULT 18 FOR AGE;
```

```sql
INSERT INTO PERSON_INFO (ID,LastName,FirstName)
VALUES (1,'PATEL','SEEMA')

SELECT * FROM PERSON_INFO

---------4) UNIQUE KEY----------------------
CREATE TABLE PERSON_MASTER (
    ID INT UNIQUE,
    LASTNAME VARCHAR(255),
    FIRSTNAME VARCHAR(255) ,
    AGE INT
);

INSERT INTO PERSON_MASTER
VALUES(1,'PATEL','BHOOMI',19)


--ERROR
INSERT INTO PERSON_MASTER
VALUES(1,'PATEL','BHOOMI',19)

----
INSERT INTO PERSON_MASTER
VALUES(NULL,'SHAH','KRISHNA',19)

--ERROR
INSERT INTO PERSON_MASTER
VALUES(NULL,'PATEL','KRISHNA',19)

DROP TABLE PERSON_MASTER

CREATE TABLE PERSON_MASTER (
   ID INT,
   LASTNAME VARCHAR(255),
   FIRSTNAME VARCHAR(255) ,
   AGE INT
);

--Create a UNIQUE KEY on the “ID" column
--when the "PERSON_MASTER" table is already created

ALTER TABLE PERSON_MASTER
ADD UNIQUE (ID);

INSERT INTO PERSON_MASTER VALUES(2,'PATEL','MITA',19)

--ERROR
INSERT INTO PERSON_MASTER VALUES(2,'PATEL','MITA',19)

DROP TABLE PERSON_MASTER
CREATE TABLE PERSON_MASTER (
    ID INT,
    LASTNAME VARCHAR(255),
    FIRSTNAME VARCHAR(255) ,
```

```
    AGE INT
);

--ADD UNIQUE CONSTRAINT ON MULTIPLE COLUMN
ALTER TABLE PERSON_MASTER
ADD UNIQUE (ID,LASTNAME);

--OR
ALTER TABLE Persons
ADD CONSTRAINT UC_Persons UNIQUE (ID,FirstName);


-------5) PRIMARY KEY CONSTRAINT-------

CREATE TABLE CUSTOMER (
    ID INT PRIMARY KEY,
    LastName varchar(255),
    FirstName varchar(255),
);

INSERT INTO CUSTOMER VALUES(1,'PATEL','BHOOMI')

--error (as ID is not unique)
INSERT INTO CUSTOMER VALUES(1,'PATEL','SITAL')

--ERROR (as ID is NULL)
INSERT INTO CUSTOMER VALUES(NULL,'PATEL','SITAL')

---Create a PRIMARY KEY on the “ID" column
--when the "CUSTOMER" table is already created.

DROP TABLE CUSTOMER


CREATE TABLE CUSTOMER (
    ID INT  not null,
    LastName varchar(255),
    FirstName varchar(255),
);

--ID COLUMN MUST BE NOT NULL
ALTER TABLE CUSTOMER
ADD PRIMARY KEY (ID);


-----------6) FORIEGN KEY CONSTRAINTS-----------------
CREATE TABLE CUSTOMER_DATA (
    ID INT PRIMARY KEY,
    LastName varchar(255),
    FirstName varchar(255),
);


CREATE TABLE ORDERS (
```

```sql
ORDERNO INT NOT NULL,
CUST_ID INT FOREIGN KEY REFERENCES CUSTOMER_DATA(ID) );


--ERROR
INSERT INTO ORDERS VALUES(1,5);


--FIRST INSERT RECORD IN CUSTOMER_DATA TABLE(parent table)
INSERT INTO CUSTOMER_DATA VALUES(5,'PATEL','SEEMA')

--NOW YOU CAN INSRT RECORD IN ORDER TABLE WITH CUSTOMER ID 5

INSERT INTO ORDERS VALUES(1,5);

----Create a FOREIGN KEY on the "CUST_ID" column
--when the "Order" table is already created.
DROP TABLE ORDERS

CREATE TABLE ORDERS (
ORDERNO INT NOT NULL,
CUST_ID INT );

ALTER TABLE ORDERS
ADD FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER_DATA(ID);
```